



UNIVERSITAS
GADJAH MADA

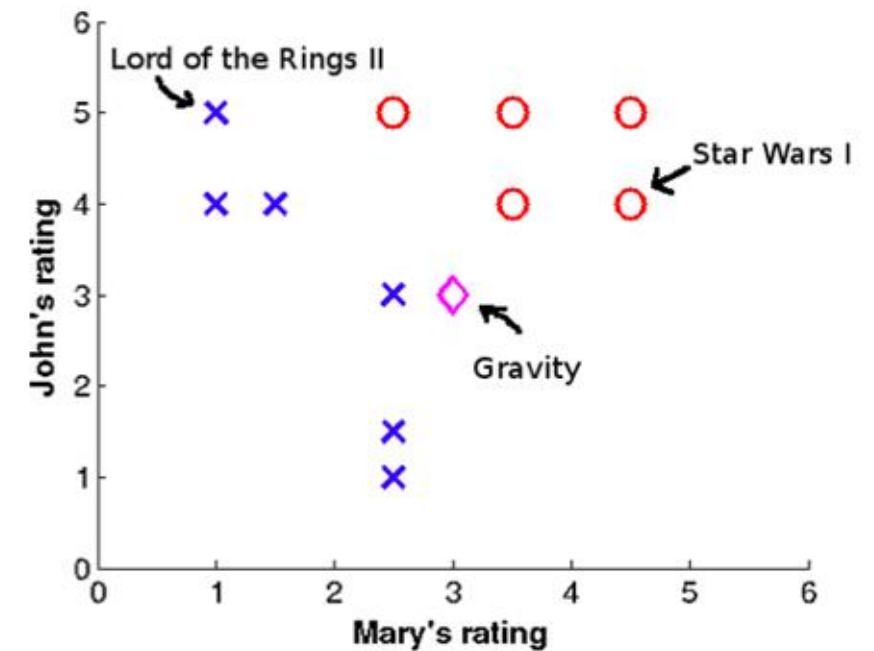


Linear Classifier – Single Layer Perceptron

Afiahayati, Ph.D.
Dept. of Computer Science and Electronics
Universitas Gadjah Mada
afia@ugm.ac.id

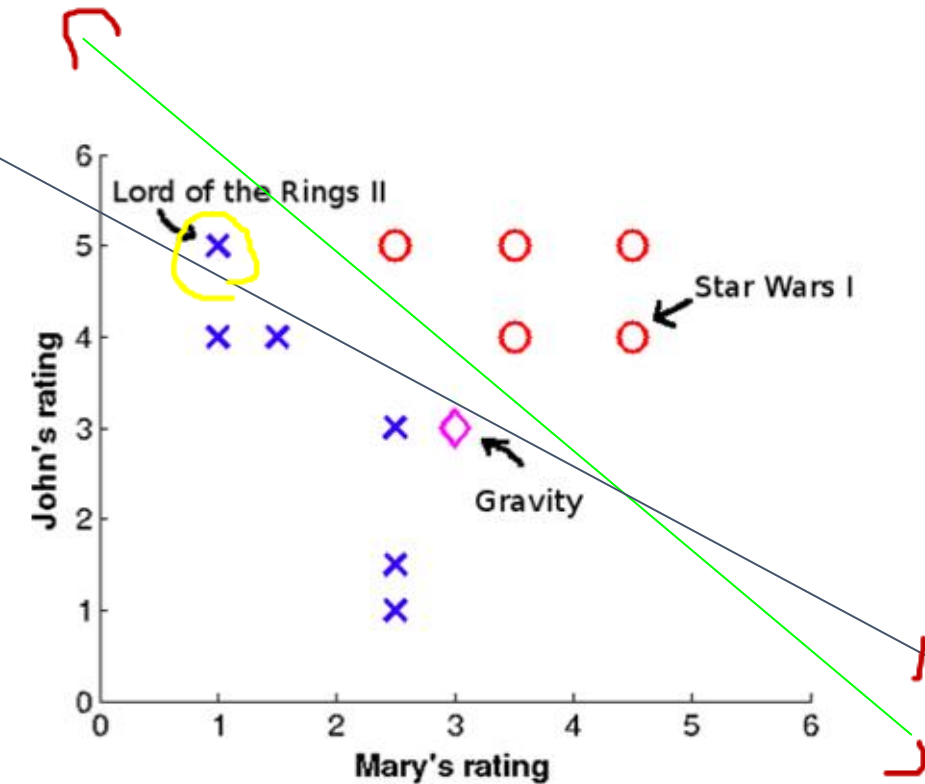
Linear Classifier

Movie name	Mary's rating	John's rating	I like?
Lord of the Rings II	1	5	No
...	...	...	...
Star Wars I	4.5	4	Yes
Gravity	3	3	?

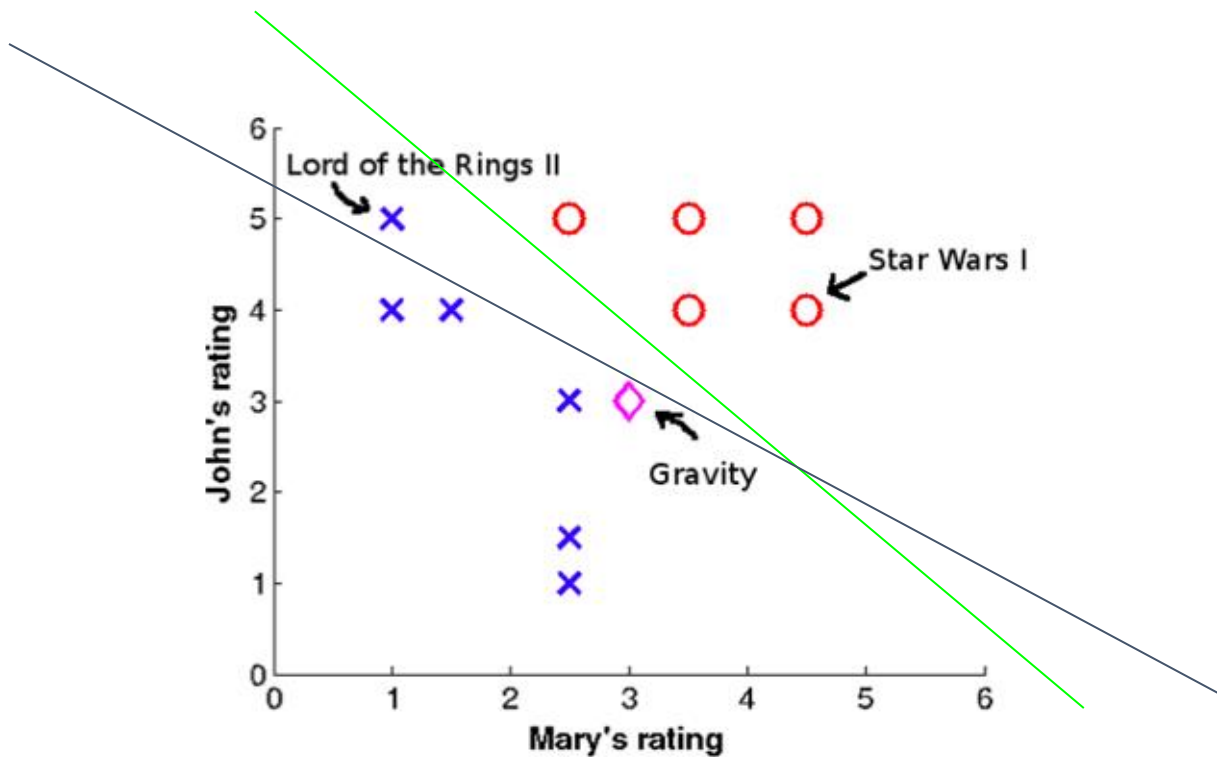


Linear Classifier

Movie name	Mary's rating	John's rating	I like?
Lord of the Rings II	1	5	No
...	...	...	...
Star Wars I	4.5	4	Yes
Gravity	3	3	?



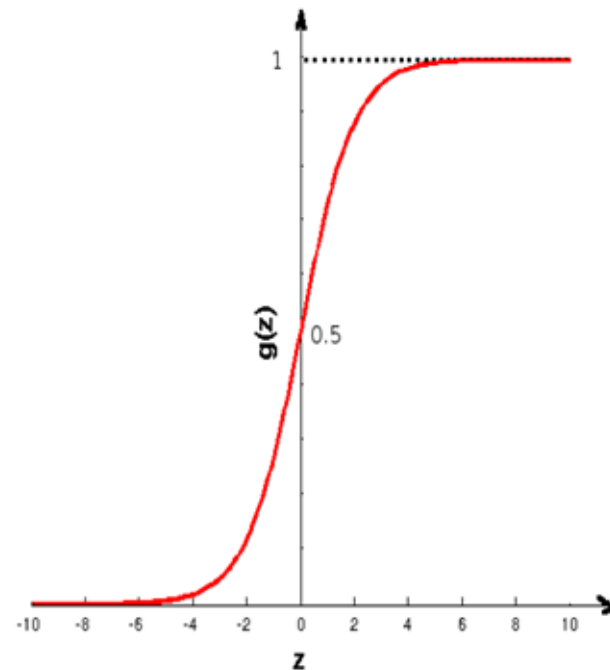
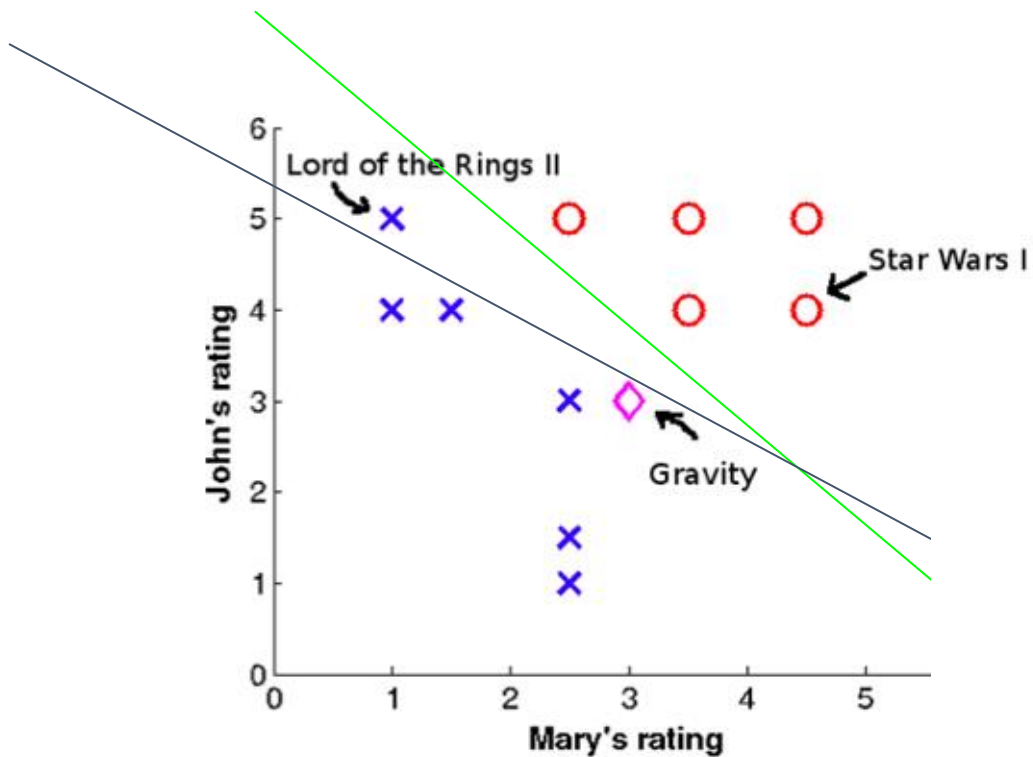
Linear Classifier



$$h(x; \theta, b) = \theta_1 x_1 + \theta_2 x_2 + b$$

$$h(x; \theta, b) = \theta^T x + b$$

Linear Classifier



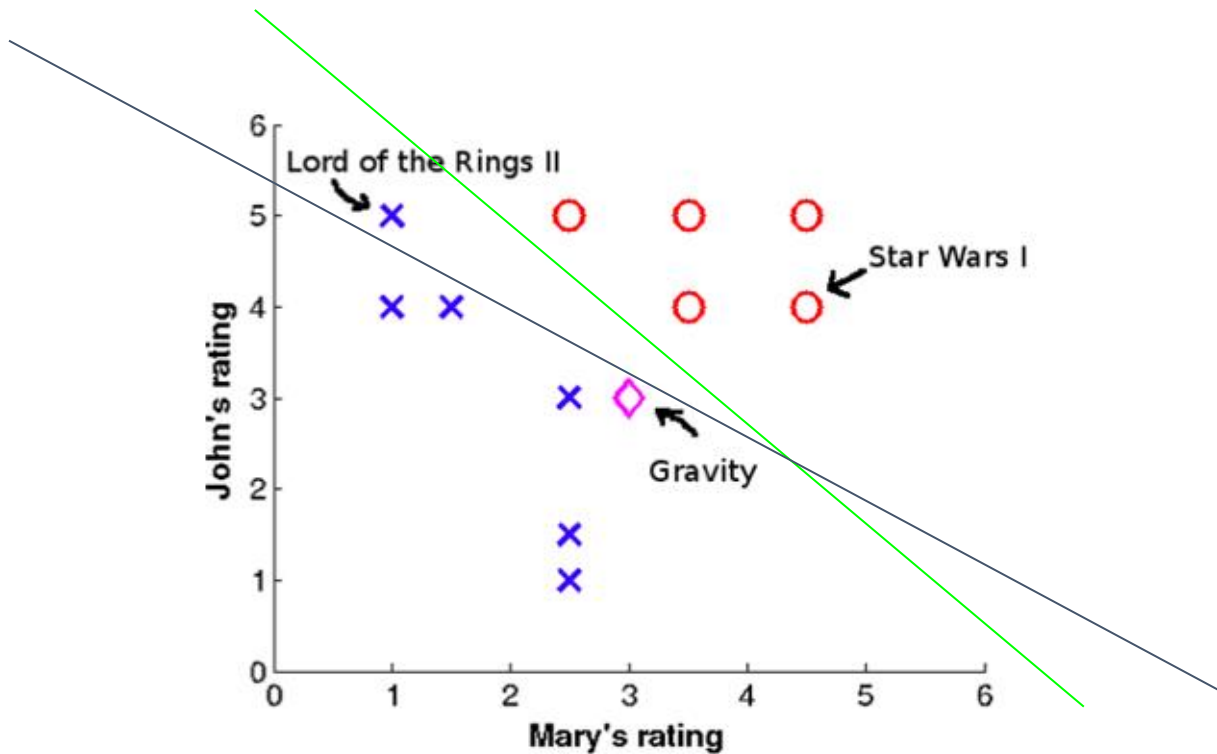
$$h(x; \theta, b) = \theta_1 x_1 + \theta_2 x_2 + b$$

$$h(x; \theta, b) = \theta^T x + b$$

$$h(x; \theta, b) = g(\theta^T x + b)$$

$$g(z) = \frac{1}{1 + \exp(-z)}$$

Linear Classifier – Single Layer Perceptron (SLP)

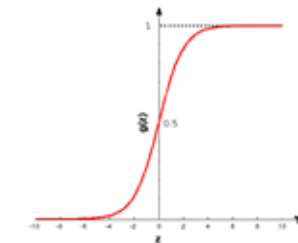
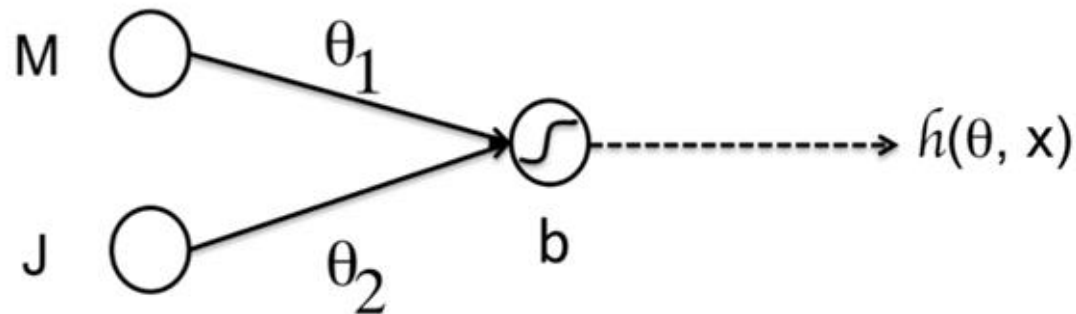


$$h(x; \theta, b) = \theta_1 x_1 + \theta_2 x_2 + b$$

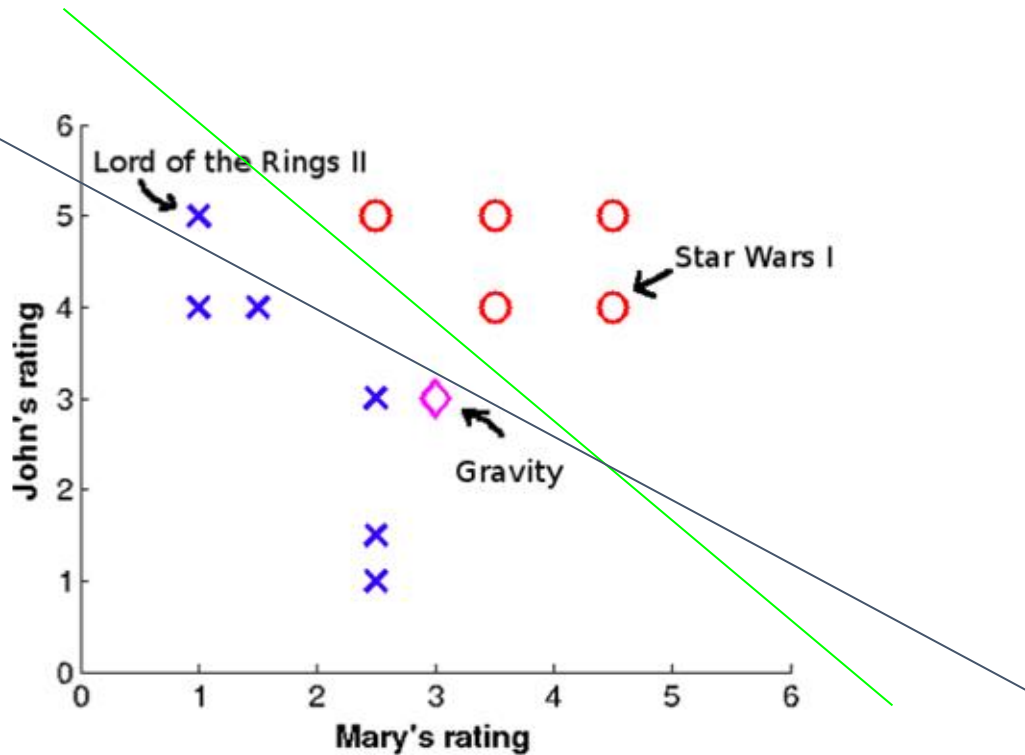
$$h(x; \theta, b) = \theta^T x + b$$

$$h(x; \theta, b) = g(\theta^T x + b)$$

$$g(z) = \frac{1}{1 + \exp(-z)}$$



Linear Classifier – Single Layer Perceptron (SLP)



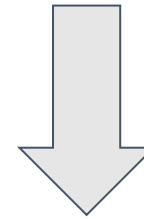
How to get
appropriate linear
classifier ?



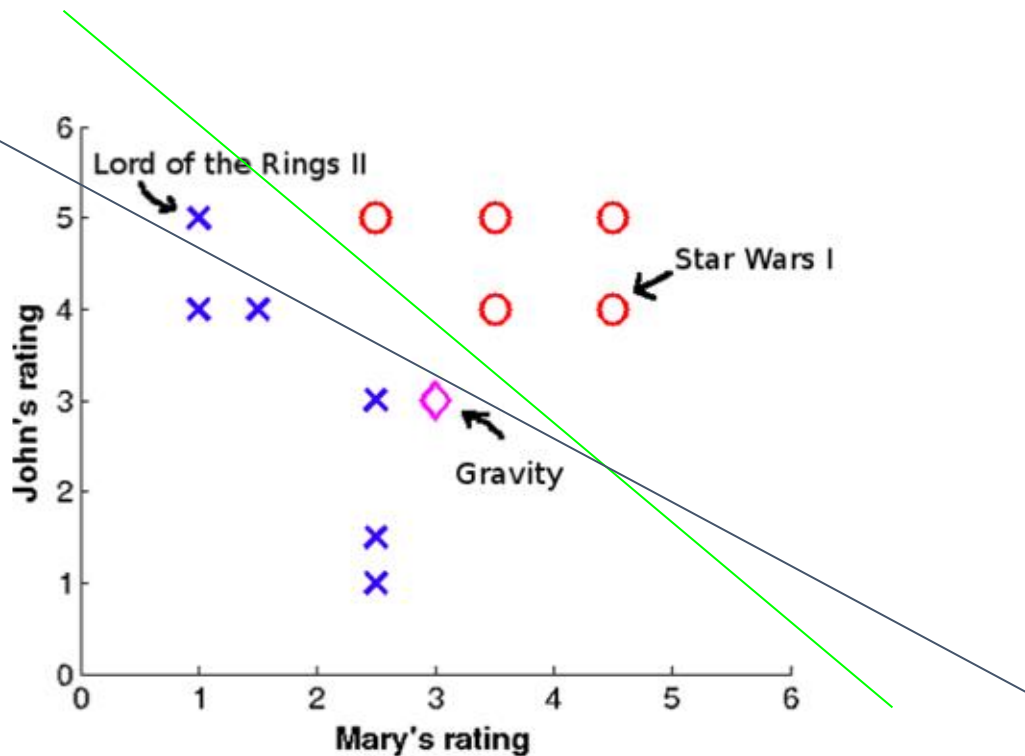
LEARNING
from Error

Single Layer Perceptron (SLP) – Stochastic Gradient Descent

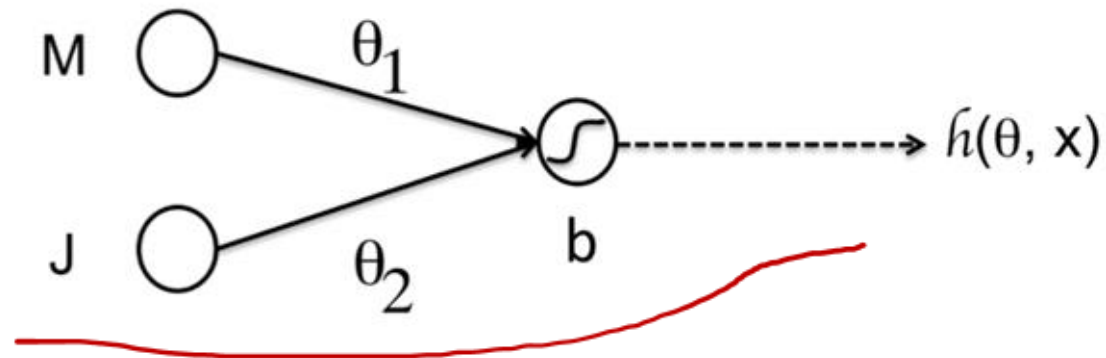
LEARNING from Error



Error function
Cost function
Lost Function



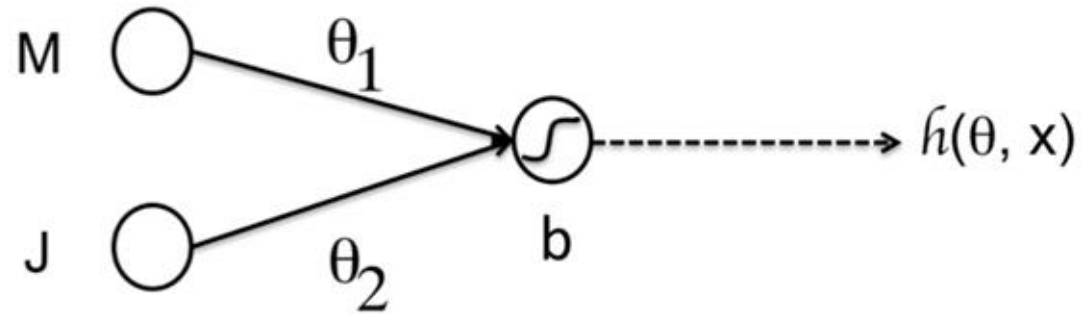
Single Layer Perceptron (SLP) – Gradient Descent



$\theta_1 = \theta_1 - \alpha \Delta \theta_1$
 $\theta_2 = \theta_2 - \alpha \Delta \theta_2$
 $b = b - \alpha \Delta b$

Handwritten red notes: "LR" with an arrow pointing to the equations, and red checkmarks under each equation.

Single Layer Perceptron (SLP) – Gradient Descent



$$\theta_1 = \theta_1 - \alpha \Delta \theta_1$$

$$\theta_2 = \theta_2 - \alpha \Delta \theta_2$$

$$b = b - \alpha \Delta b$$

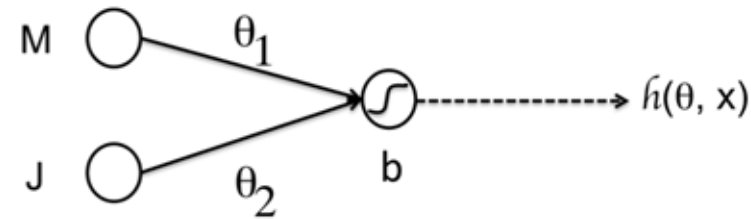


$$\begin{aligned} \Delta \theta_1 &= \frac{\partial}{\partial \theta_1} \left(h(x^{(i)}; \theta, b) - y^{(i)} \right)^2 \\ &= 2 \left(h(x^{(i)}; \theta, b) - y^{(i)} \right) \frac{\partial}{\partial \theta_1} h(x^{(i)}; \theta, b) \\ &= 2 \left(g(\theta^T x^{(i)} + b) - y^{(i)} \right) \frac{\partial}{\partial \theta_1} g(\theta^T x^{(i)} + b) \end{aligned}$$

Single Layer Perceptron (SLP) – Gradient Descent



UNIVERSITAS
GADJAH MADA



$$\begin{aligned}\Delta\theta_1 &= \frac{\partial}{\partial\theta_1} \left(h(x^{(i)}; \theta, b) - y^{(i)} \right)^2 \\ &= 2 \left(h(x^{(i)}; \theta, b) - y^{(i)} \right) \frac{\partial}{\partial\theta_1} h(x^{(i)}; \theta, b) \\ &= 2 \left(g(\theta^T x^{(i)} + b) - y^{(i)} \right) \frac{\partial}{\partial\theta_1} g(\theta^T x^{(i)} + b)\end{aligned}$$

$$\frac{\partial g}{\partial z} = [1 - g(z)]g(z)$$

$$\begin{aligned}\frac{\partial}{\partial\theta_1} g(\theta^T x^{(i)} + b) &= \frac{\partial g(\theta^T x^{(i)} + b)}{\partial(\theta^T x^{(i)} + b)} \frac{\partial(\theta^T x^{(i)} + b)}{\partial\theta_1} \\ &= [1 - g(\theta^T x^{(i)} + b)] g(\theta^T x^{(i)} + b) \frac{\partial(\theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + b)}{\partial\theta_1} \\ &= [1 - g(\theta^T x^{(i)} + b)] g(\theta^T x^{(i)} + b) x_1^{(i)}\end{aligned}$$

Single Layer Perceptron (SLP) – Gradient Descent

$$\begin{aligned}\Delta\theta_1 &= \frac{\partial}{\partial\theta_1} \left(h(x^{(i)}; \theta, b) - y^{(i)} \right)^2 \\ &= 2 \left(h(x^{(i)}; \theta, b) - y^{(i)} \right) \frac{\partial}{\partial\theta_1} h(x^{(i)}; \theta, b) \\ &= 2 \left(g(\theta^T x^{(i)} + b) - y^{(i)} \right) \frac{\partial}{\partial\theta_1} g(\theta^T x^{(i)} + b)\end{aligned}$$

$$\begin{aligned}\Delta\theta_1 &= 2[g(\theta^T x^{(i)} + b) - y^{(i)}] [1 - g(\theta^T x^{(i)} + b)] g(\theta^T x^{(i)} + b) x_1^{(i)} \\ \Delta\theta_2 &= 2[g(\theta^T x^{(i)} + b) - y^{(i)}] [1 - g(\theta^T x^{(i)} + b)] g(\theta^T x^{(i)} + b) x_2^{(i)} \\ \Delta b &= 2[g(\theta^T x^{(i)} + b) - y^{(i)}] [1 - g(\theta^T x^{(i)} + b)] g(\theta^T x^{(i)} + b)\end{aligned}$$

$$\begin{aligned}\frac{\partial}{\partial\theta_1} g(\theta^T x^{(i)} + b) &= \frac{\partial g(\theta^T x^{(i)} + b)}{\partial(\theta^T x^{(i)} + b)} \frac{\partial(\theta^T x^{(i)} + b)}{\partial\theta_1} \\ &= [1 - g(\theta^T x^{(i)} + b)] g(\theta^T x^{(i)} + b) \frac{\partial(\theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + b)}{\partial\theta_1} \\ &= [1 - g(\theta^T x^{(i)} + b)] g(\theta^T x^{(i)} + b) x_1^{(i)}\end{aligned}$$

Single Layer Perceptron (SLP) – Gradient Descent

$$\Delta\theta_1 = 2[g(\theta^T x^{(i)} + b) - y^{(i)}][1 - g(\theta^T x^{(i)} + b)]g(\theta^T x^{(i)} + b)x_1^{(i)}$$

$$\Delta\theta_2 = 2[g(\theta^T x^{(i)} + b) - y^{(i)}][1 - g(\theta^T x^{(i)} + b)]g(\theta^T x^{(i)} + b)x_2^{(i)}$$

$$\Delta b = 2[g(\theta^T x^{(i)} + b) - y^{(i)}][1 - g(\theta^T x^{(i)} + b)]g(\theta^T x^{(i)} + b)$$

1. Initialise the weight and bias randomly
2. Calculate the error function, if error still big then do
 - a. Update the weight
 - b. Update the bias

Hands on – Iris Dataset

- <https://www.kaggle.com/datasets/uciml/iris>
- 3 class @50 => total : 150 data
- #feature = 4 features
- Training = 120
- Testing = 30 (10 data terakhir dari setiap kelasnya)
- Define an architecture of SLP

Hands on – Iris Dataset - spreadsheet

- <https://www.kaggle.com/datasets/uciml/iris>
- 2 class @50 => total : 100 data
 - Iris Setosa
 - Iris Versicolour
- #feature = 4 features
- Training = 80
- Testing = 20
- Define and architecture of SLP
 - Output neuron => binary encoding

Hands on – Iris Dataset - spreadsheet

- Spreadsheet

<https://docs.google.com/spreadsheets/d/1yhi8ASebpmljJ3NpYQ7NmxlWLmhGOl6W/edit?usp=sharing&oid=117122342833524350176&rtpof=true&sd=true>